As **anotações em Java** (`annotations`) são uma forma de **metadados** — ou seja, informações adicionais inseridas no código-fonte que descrevem classes, métodos, variáveis, parâmetros ou pacotes. [1, 2]

Elas são sempre precedidas pelo caractere `@` (por exemplo: `@Override`). [1]

O que elas fazem?

- **Não alteram a lógica direta** do código por si sós; em vez disso, funcionam como "bilhetes" ou "etiquetas" informativas.
- **São lidas por terceiros:** Podem ser interpretadas pelo **compilador Java**, por ferramentas de análise de código ou por **frameworks** (como Spring, Hibernate e JUnit) em tempo de execução. [1, 2, 3, 4, 5]

Principais Categorias

1. **Anotações Nativas (Built-in):** Já vêm embutidas no pacote `java.lang` do Java:
   - `@Override` : Indica que o método está sobrescrevendo um método da classe pai.
   - `@Deprecated` : Marca um método ou classe como obsoleto, desencorajando seu uso.
   - `@SuppressWarnings` : Solicita que o compilador ignore determinados avisos (warnings). [1, 2]
2. **Anotações de Frameworks:** Amplamente usadas para configuração simplificada (muitas vezes substituindo arquivos XML antigos):
   - `@Service` ou `@RestController` (no Spring).
   - `@Entity` e `@Table` (no Hibernate / JPA para mapear tabelas de banco de dados). [1, 2, 3, 4]
3. **Anotações Personalizadas (Custom):** Criadas pelo próprio desenvolvedor utilizando a sintaxe `@interface` . [1, 2]

Como elas funcionam na prática

Exemplo básico com `@Override` :

java

```java
public class Exemplo {
    @Override
    public String toString() {
        return "Minha String customizada";
    }
}
```

Use o código com cuidado.

Se você escrever a assinatura do método errado em vez de sobrescrever de verdade, o compilador apontará um erro graças à anotação `@Override`. [1]

Gostaria de saber como **criar a sua própria anotação personalizada** em Java ou como utilizá-las com **Reflection** em tempo de execução?

- DevMedia

  Entendendo Anotações em Java - DevMedia

  Entendendo Anotações em Java. O artigo trata de metadados – anotações em Java – implementados a partir do Java 1.5. Anotações são ...

  

- DIO

  Entendendo Anotações em Java | Francisco Oliveira - DIO

  6 de jul. de 2024 — Annotations são uma forma de marcar ou anotar o código fonte com informações adicionais. Elas são precedidas pelo símbolo @ e pode...

  

- Stack Overflow em Português

  java - O que são anotações e qual sua definição?

  24 de nov. de 2022 — Anotações são metadados do programa (são ortogonais a ele). Segundo a especificação da linguagem, uma anotação é um "tipo especial...

  

Mostrar tudo

gostaria de saber ambos conceitos

Para entender as **anotações personalizadas** e o **Reflection**, pense nelas como uma dupla dinâmica: com a anotação você coloca uma "etiqueta" no código, e com o Reflection você consegue ler e tomar decisões baseadas nessa etiqueta enquanto o programa está rodando.

---

1. Criando Anotações Personalizadas (Custom Annotations)

Para criar sua própria anotação, você usa a palavra-chave `@interface`. Além disso, você precisa usar duas anotações do próprio Java para definir como ela vai se comportar:

- **`@Target`**: Define *onde* sua anotação pode ser usada (em métodos, classes, campos, etc.).
- **`@Retention`**: Define *até quando* ela fica guardada. Para ler com Reflection, precisamos usar `RetentionPolicy.RUNTIME`.

Exemplo Prático: Vamos criar uma anotação chamada `@GarantirMaioridade`.

java

```java
import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;

@Target(ElementType.FIELD) // Só pode ser usada em variáveis/atributos (campos)
@Retention(RetentionPolicy.RUNTIME) // Fica disponível em tempo de execução
public @interface GarantirMaioridade {
    String mensagem() default "A idade mínima permitida é 18 anos!"; // Parâmetro opcional
}
```

Use o código com cuidado.

Para usar, basta colocá-la em cima de um atributo:

java

```java
public class Usuario {
    private String nome;

    @GarantirMaioridade
    private int idade;

    public Usuario(String nome, int idade) {
        this.nome = nome;
        this.idade = idade;
    }
    // Getters e Setters...
}
```

Use o código com cuidado.

---

2. O que é e como usar Reflection?

Reflection (Reflexão) é uma API do Java que permite ao seu programa **inspecionar a si mesmo** em tempo de execução. Com ela, você consegue descobrir quais métodos uma classe tem, acessar atributos privados e, claro, **ler as anotações** do código.

É assim que os grandes frameworks funcionam por debaixo dos panos.

Juntando os dois mundos: Validando a anotação com Reflection

Veja como criaríamos um "validador" que lê a nossa anotação `@GarantirMaioridade` e impede o cadastro se o usuário for menor de idade:

java

```
import java.lang.reflect.Field;

public class Validador {
    public static void validar(Object obj) throws IllegalAccessException {
        // Obtém a classe do objeto dinamicamente
        Class<?> classe = obj.getClass();

        // Percorre todos os atributos (campos) da classe
        for (Field campo : classe.getDeclaredFields()) {

            // Verifica se o campo possui a nossa anotação personalizada
            if (campo.isAnnotationPresent(GarantirMaioridade.class)) {

                campo.setAccessible(true); // Permite ler o campo mesmo se for
privado
                int idade = (int) campo.get(obj); // Pega o valor do campo

                if (idade < 18) {
```

Use o código com cuidado.

Testando a execução:

java

```
public class Principal {
    public static void main(String[] args) {
        Usuario user1 = new Usuario("Carlos", 16);

        try {
            Validador.validar(user1);
            System.out.println("Usuário validado com sucesso!");
        } catch (Exception e) {
            // Vai imprimir: "A idade mínima permitida é 18 anos!"
            System.out.println("Erro de validação: " + e.getMessage());
        }
```

```
        }
    }
```

Use o código com cuidado.

Resumo dos Benefícios

- **Código mais limpo:** Você separa as regras de validação ou configuração da lógica de negócios da classe.
- **Automação:** O Reflection permite criar utilitários genéricos que funcionam para qualquer classe que use suas anotações.